

Part A — Core SQL Queries

A1 Monthly Revenue Trend

Calculate the total revenue (sum of payment_value) per month, ordered chronologically. Include the month-over-month percentage change in revenue. Display: year, month, total_revenue, prev_month_revenue, mom_change_pct.

Answer :

```
with
  revenue as (select
    extract(year from o.order_delivered_customer_date) as year -- final date for revenue &
    extraction to number for order data
    , extract(month from o.order_delivered_customer_date) as month
    , round(sum(op.payment_value)::decimal, 2) as total_revenue
  from orders o -- base table
    left join order_payments op on o.order_id = op.order_id -- taking payment_value for
revenue
    where o.order_status = 'delivered'
    group by 1, 2
    order by 1,2 asc)
, prev_revenue as (select
  r.*
  , coalesce(lag(total_revenue, 1) over (order by r.year, r.month asc),0) as
prev_month_revenue -- taking previous month revenue
  from revenue r)
select
  pr.year
, pr.month
, pr.total_revenue
, pr.prev_month_revenue
, coalesce(round(((pr.total_revenue - pr.prev_month_revenue) / nullif(pr.prev_month_revenue,0) *
100)::decimal,2),0) as mom_change_pct -- Get MoM movement in percentage with handling err divided 0
from prev_revenue pr
order by 1, 2 asc
```

Result :

	year	month	total_revenue	prev_month_revenue	mom_change_pct
1	2016	10	34953.78	0	0
2	2016	11	11376.19	34953.78	-67.45
3	2016	12	960.85	11376.19	-91.55
4	2017	1	38708.63	960.85	3928.58
5	2017	2	228152.25	38708.63	489.41
6	2017	3	387275.32	228152.25	69.74
7	2017	4	307722.84	387275.32	-20.54
8	2017	5	600917.76	307722.84	95.28

A2 Top 10 Product Categories by Revenue

Identify the top 10 product categories (in English) ranked by total revenue. Include the

number of distinct orders, total items sold, and average order value per category. Exclude cancelled orders.

Answer :

```
with
  category_metrics as (select
    -- Menggunakan coalesce agar jika translasi Inggris tidak ketemu, tetap menampilkan
    nama asli
    , initcap(replace(coalesce(t.product_category_name_english, p.product_category_name),
    '-',
    ' ')) as product_category_name
    , count(distinct o.order_id) as num_orders
    , count(i.order_item_id) as total_items_sold
    , sum(op.payment_value) as total_revenue
  from orders o
    inner join order_items i on o.order_id = i.order_id
    inner join products p on i.product_id = p.product_id
    left join order_payments op on o.order_id = op.order_id
    left join product_category_name_translation t -- taking for product name in
    english
    on p.product_category_name = t.product_category_name
  where o.order_status != 'canceled'
    and p.product_category_name is not null
  group by p.product_category_name, t.product_category_name_english)
select
  product_category_name
  , num_orders
  , total_items_sold
  , ROUND(total_revenue::DECIMAL, 2) as total_revenue
  , ROUND((total_revenue / num_orders)::DECIMAL, 2) as average_order_value -- calculate AOV
from category_metrics
order by total_revenue desc
limit 10;
```

Result :

	product_category_name	num_orders	total_items_sold	total_revenue	average_order_value
1	Bed Bath Table	9399	11805	1711258.08	182.07
2	Health Beauty	8800	9935	1653730.45	187.92
3	Computers Accessories	6654	8036	1571543.81	236.18
4	Furniture Decor	6425	8707	1424782.52	221.76
5	Watches Gifts	5604	6180	1421715.28	253.7
6	Sports Leisure	7673	8893	1381363.23	180.03
7	Housewares	5847	7296	1086565.32	185.83
8	Auto	3873	4349	843343.61	217.75
9	Garden Tools	3505	4554	823517.8	234.96
10	Cool Stuff	3617	3970	759735.08	210.05

A3 Customer Cohort Retention

Group customers by their first purchase month (acquisition cohort). For each cohort, calculate how many customers made at least one additional purchase in the following 1, 2, and 3 months. Display cohort_month, cohort_size, retained_m1, retained_m2, retained_m3.

Answer :

```

/*
Goals
- cohort_month: The month in which a cohort of customers made their first purchase (acquisition month).
- cohort_size: The total number of new unique customers acquired in that month.
- retained_m1 (Month 1): The number of customers from that cohort who returned and made another purchase in the
next month following their first purchase.
- retained_m2 (Month 2): The number of customers who returned in the two months after following their first
purchase.
- retained_m3 (Month 3): The number of customers who returned in the three months after following their first
purchase.
*/
with
  customer_revenue as ( -- taking all success trx per customer in every month
    select
      c.customer_unique_id
      , o.order_id
      , o.order_purchase_timestamp
      , date_trunc('month', o.order_purchase_timestamp) as order_month
    from orders o
      left join customers c on o.customer_id = c.customer_id
      where o.order_status != 'canceled')
  , cohort_definition as ( -- taking the month of the first purchase (Acquisition Month) for each customer
    select
      customer_unique_id
      , MIN(order_month) as cohort_month
    from customer_revenue
    group by customer_unique_id)
  , cohort_sizes as ( -- calculate the total unique customers in each cohort (Cohort Size)
    select
      cohort_month
      , COUNT(distinct customer_unique_id) as cohort_size
    from cohort_definition
    group by cohort_month)
  , retention_intervals as (-- calculate the number of months between the next purchase and the first month of
shopping.
    select
      r.customer_unique_id
      , cd.cohort_month
      , (extract(year from r.order_month) - extract(year from cd.cohort_month)) * 12 + (extract(month from
r.order_month) -
      extract(month from cd.cohort_month)) as months_next_order -- calculate months_next_order
    from customer_revenue r
      join cohort_definition cd on r.customer_unique_id = cd.customer_unique_id)
-- compile all data and pivot using case when
select
  to_char(cs.cohort_month, 'YYYY-MM') as cohort_month
  , cs.cohort_size
  , count(distinct case when ri.months_next_order = 1 then ri.customer_unique_id end) as retained_m1
  , count(distinct case when ri.months_next_order = 2 then ri.customer_unique_id end) as retained_m2
  , count(distinct case when ri.months_next_order = 3 then ri.customer_unique_id end) as retained_m3
from cohort_sizes cs
  left join retention_intervals ri on cs.cohort_month = ri.cohort_month
group by cs.cohort_month, cs.cohort_size
order by cs.cohort_month asc;

```

Result :

	cohort_month	cohort_size	retained_m1	retained_m2	retained_m3
1	2016-09	2	0	0	0
2	2016-10	297	0	0	0
3	2016-12	1	1	0	0
4	2017-01	762	3	2	1
5	2017-02	1735	4	5	2
6	2017-03	2603	13	9	10
7	2017-04	2334	14	5	4
8	2017-05	3571	17	18	14
9	2017-06	3126	14	11	13
10	2017-07	3868	20	13	10
11	2017-08	4162	28	14	11
12	2017-09	4112	28	22	12
13	2017-10	4446	31	11	4
14	2017-11	7270	40	28	12
15	2017-12	5479	14	15	19
16	2018-01	6992	23	26	20

Part B — Query Optimisation

Bottleneck :

- Duplicate processing (join and rejoining again same table): part table products and order_item scanned 3 times in subqueries (b, ps, and ls) from 3 of subquery can be put into 1 CTE (this is impacting to memory executing database)
- Duplicate in processing order_items which merge products, order_items and order_payments, since 1 order can have several order item. Joining between order_items and order_payments without any aggregation in order first can impact multiple data rows. Which payment_value will repeat calculation turns into duplicate if 1 order have more than 1 item
- Early aggregation (doing group by product_category_name) in every subquery that forcing db doing operation sort repetition before data is in compile in main query / master query

How to Optimize it:

- Use CTE for taking products and order (only 1 time)
- Staging payment table : aggregate data credit_card and boleto in level order_id before join to avoid duplicate
- From 2 point above Window Function will be optimized since the data already clean (no duplicate)
- Indexing recommendation, ensure *foreign key / unique key / surge key* containing index (those index can be use as sort_key for optimizing table run so users no need do sort again for exploration):
 - order_items.order_id & order_item.product_id
 - order_payments.order_id
 - orders.customer_id & orders.order_delivered_customer_date

Query Optimized:

```
with
  payment_summary as (-- aggregate payment per order_id first to ensure no duplicate order_id causes
order_items to be counted twice
    select
        order_id
        , sum(payment_value) filter (where payment_type = 'credit_card') as credit_card_revenue
        , sum(payment_value) filter (where payment_type = 'boleto') as boleto_revenue
    from order_payments
    group by order_id
  , base_metrics as ( -- base CTE for joining to another CTE
    select
        t.product_category_name_english as category
        , COUNT(distinct o.order_id) as total_orders
```

```

, sum(oi.price) as total_revenue
, sum(pay.credit_card_revenue) as credit_card_revenue
, sum(pay.boleto_revenue) as boleto_revenue
-- calculate late delivered
, COUNT(distinct case
            when o.order_delivered_customer_date > o.order_estimated_delivery_date
            then o.order_id end) as late_count
, AVG(case
        when o.order_delivered_customer_date > o.order_estimated_delivery_date
        then EXTRACT(epoch from (o.order_delivered_customer_date - o.order_estimated_delivery_date))
/
            86400.0
        end) as avg_days_late
from order_items oi
    join products p on oi.product_id = p.product_id
    join product_category_name_translation t on t.product_category_name = p.product_category_name
    join orders o on o.order_id = oi.order_id
    left join payment_summary pay on pay.order_id = o.order_id
where o.order_delivered_customer_date is not null
group by t.product_category_name_english)
, ranked_metrics as (-- Calculate rank and percentage
select
    rank() over (order by total_revenue desc) as revenue_rank
, category
, total_orders
, total_revenue
-- Revise in this part: direct divide cause total_orders already fixed from previous CTE
, round((total_revenue / NULLIF(total_orders, 0))::numeric, 2) as avg_order_value
, credit_card_revenue
, boleto_revenue
, round((credit_card_revenue * 100.0 / NULLIF(credit_card_revenue + boleto_revenue, 0))::numeric,
1) as credit_card_share_pct
, late_count
, round((late_count * 100.0 / NULLIF(total_orders, 0))::numeric, 1) as late_rate_pct
, round(avg_days_late::numeric, 1) as avg_days_late
, rank() over (order by (late_count * 100.0 / NULLIF(total_orders, 0)) desc) as late_rank
from base_metrics)
select
    revenue_rank
, category
, total_orders
, round(total_revenue::numeric, 2) as total_revenue
, round((total_revenue * 100.0 / NULLIF(sum(total_revenue) over (), 0))::numeric, 2) as revenue_share_pct
, round((sum(total_revenue) over (
    order by total_revenue desc
    rows between unbounded preceding and current row
) * 100.0 / NULLIF(sum(total_revenue) over (), 0))::numeric, 2) as cumulative_revenue_pct
, avg_order_value
, round(credit_card_revenue::numeric, 2) as credit_card_revenue
, round(boleto_revenue::numeric, 2) as boleto_revenue
, credit_card_share_pct
, late_count
, late_rate_pct
, avg_days_late
, late_rank
from ranked_metrics
order by revenue_rank;

```

Result :

	revenue_rank	category	total_orders	total_revenue	revenue_share_pct	cumulative_revenue_pct	
1	1	health_beauty	8649	1233211.61	9.45	9.45	
2	2	watches_gifts	5493	1165898.98	8.94	18.39	
3	3	bed_bath_table	9272	1023434.76	7.85	26.24	
4	4	sports_leisure	7530	954695.05	7.32	33.56	
5	5	computers_accessories	6529	888613.62	6.81	40.37	
6	6	furniture_decor	6307	711927.69	5.46	45.82	
7	7	housewares	5743	615628.69	4.72	50.54	
8	8	cool_stuff	3559	610204.1	4.68	55.22	
9	9	auto	3809	578849.35	4.44	59.66	

	avg_order_value	credit_card_revenue	boleto_revenue	credit_card_share_pct	late_count	late_rate_pct	
	142.58	1297876.65	273884.88	82.6	776	9	
	212.25	1132730.51	203631.27	84.8	468	8.5	
	110.38	1369480.82	257884.53	84.2	811	8.7	
	126.79	1053542.67	246842.8	81	584	7.8	
	136.1	955849.83	554365.99	63.3	503	7.7	
	112.88	1052492.41	298682.23	77.9	535	8.5	
	107.2	797360.17	228186.87	77.7	399	6.9	
	171.45	585406.37	137658.96	81	243	6.8	
	151.97	679495.44	129258.66	84	328	8.6	

avg_days_late	late_rank
8.8	14
9.1	20
9.8	16
9.8	31
9.4	32
9.5	21
8.9	40
10.5	44
11	18

Or Result in CSV - UTF-8 (sample) :

revenue_rank,category,total_orders,total_revenue,revenue_share_pct,cumulative_revenue_pct,
avg_order_value,credit_card_revenue,boleto_revenue,credit_card_share_pct,late_count,late_rate_pct,avg_days_late,late_rank

1,health_beauty,8649,1233211.61,9.45,9.45,142.58,1297876.65,273884.88,82.6,776,9,8.8,14
2,watches_gifts,5493,1165898.98,8.94,18.39,212.25,1132730.51,203631.27,84.8,468,8.5,9.1,20
3,bed_bath_table,9272,1023434.76,7.85,26.24,110.38,1369480.82,257884.53,84.2,811,8.7,9.8,16
4,sports_leisure,7530,954695.05,7.32,33.56,126.79,1053542.67,246842.8,81,584,7.8,9.8,31
5,computers_accessories,6529,888613.62,6.81,40.37,136.1,955849.83,554365.99,63.3,503,7.7,9.4,32
6,furniture_decor,6307,711927.69,5.46,45.82,112.88,1052492.41,298682.23,77.9,535,8.5,9.5,21
7,housewares,5743,615628.69,4.72,50.54,107.2,797360.17,228186.87,77.7,399,6.9,8.9,40
8,cool_stuff,3559,610204.1,4.68,55.22,171.45,585406.37,137658.96,81,243,6.8,10.5,44
9,auto,3809,578849.35,4.44,59.66,151.97,679495.44,129258.66,84,328,8.6,11,18
10,toys,3804,471197.48,3.61,63.27,123.87,489487.88,97229.12,83.4,286,7.5,10.1,35